

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

Members:

YADAV Anshuman Krishna (exercise 4, 5 & 6)

MAHALINGAM Nithees (exercise 1, 2 & 3)

SARAVANAN Arun Prasath (Absent)

Exercise 1:

Algorithm ReverseInteger(n)

Input: Non-negative integer n

Output: Integer formed by reversing digits of n

Set reversed <- 0

While n > 0 do

 digit <- n mod 10

 reversed <- (reversed × 10) + digit

 n <- n div 10

End While

Return reversed

Complexity Analysis Questions:

1. How many arithmetic operations are performed for a d-digit number?

Answer:

Suppose the number has **n** digits.

Each loop iteration performs:

- 1 modulo (%)
- 1 multiplication (*)
- 1 addition (+)
- 1 division (//)

So the total operations are : **4n**

2. What is the time complexity (number of expensive operations) in terms of the input value n?

Answer:

Each digit is processed once.

If the input digit has **n** digits, then:

T(n) = O(n)

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

So the time complexity is : $O(n)$

Exercise 2:

Algorithm IsBalanced(s)

Input: String s containing (), [], {}

Output: True if balanced, False otherwise

Create empty stack S

For each character ch in s do

 If ch is '(' or '[' or '{' then

 Push ch onto stack S

 Else // ch is closing symbol

 If stack S is empty then

 Return False

 top <- Pop from stack S

 If ch is ')' and top ≠ '(' then

 Return False

 If ch is ']' and top ≠ '[' then

 Return False

 If ch is '}' and top ≠ '{' then

 Return False

If stack S is empty then

 Return True

Else

 Return False

Complexity Analysis Questions:

1. How many comparisons are needed for an n-character string?

Answer:

For each character:

- We check whether it is an opening bracket.
- If it is a closing bracket, we:
 - Check if the stack is empty.
 - Check if it matches the correct opening bracket on top of the stack.

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

All of these checks take a constant amount of time for each character (at most a few comparisons).

So, for a string of n characters, we do this same small amount of work n times.

That means the total number of operations grows proportionally with n .

Therefore, the total time complexity is : $O(n)$

2. What happens if we extend to include additional symbol pairs or other different characters?

Answer:

If we add more symbol pairs, nothing changes structurally.

Only change is:

- Adding more matching conditions.

The time complexity remains the same i.e. $O(n)$.

Exercise 3:

Algorithm MergeIntervals(intervals)

Input: List of intervals [start, end]

Output: List of merged non-overlapping intervals

If intervals is empty

 Return empty list

Sort intervals based on start value

Create empty list result

Set current $\leftarrow$ intervals[0]

For each interval next from intervals[1] to end do

 If next.start $\leq$ current.end then

 current.end $\leftarrow$ max(current.end, next.end)

 Else

 Add current to result

 current $\leftarrow$ next

Add current to result

Return result

Complexity Analysis Questions:

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

1. How many comparison operations are needed for n intervals?

Answer:

There are 2 parts:

a) Sorting:

Sorting takes : $n \log n$

b) Merging Loop:

We iterate once through intervals : $n-1$, so n

The total comparisons : $n \log n + n = n \log n$

2. What is the time complexity of your solution?

Answer:

- Sorting : $O(n \log n)$
- Merging pass : $O(n)$

Overall time complexity : $O(n \log n)$

Exercise 4:

```
FUNCTION EvaluatePolynomial(coeffs, x) -> Real
result <- 0
```

```
FOR i <- length(coeffs) - 1 DOWNT0 0 DO
result <- result * x + coeffs[i]
END FOR
```

```
RETURN result
```

Complexity Analysis Questions:

1. How many multiplications are needed for a degree- n polynomial?

Answer:

n multiplications

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

2. Compare with the naive approach: what is the improvement?

Answer:

Naive approach: Calculating x^k takes k multiplications. Summing that up for all terms gives roughly $n^2/2$ multiplications, hence: $O(n^2)$

Horner's approach: Uses n multiplications, hence, $O(n)$

Improvement: Reduces the complexity by a lot for large polynomials (from Quadratic to Linear)

Exercise 5:

Approach A: Using Temporary Array (Linear Space)

```
PROCEDURE RotateTemp(Vect, k)
n <- length(Vect)
k <- k mod n
temp <- empty list
```

```
FOR i <- n - k TO n - 1 DO
temp.Add(Vect[i])
END FOR
```

```
FOR i <- 0 TO n - k - 1 DO
temp.Add(Vect[i])
END FOR
```

```
Vect <- temp
```

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$

Approach B: Rotate One by One (Related to $n*k$ time)

```
PROCEDURE RotateOneByOne(Vect, k)
n <- length(Vect)
```

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

```
k <- k mod n
```

```
FOR i <- 1 TO k DO
```

```
last <- Vect[n - 1]
```

```
FOR j ← n - 1 DOWNT0 1 DO
```

```
Vect[j] ← Vect[j - 1]
```

```
END FOR
```

```
Vect[0] <- last
```

```
END FOR
```

- Time Complexity: $O(n \cdot k)$
- Space Complexity: $O(1)$

Approach C: Reverse Segments Method (Linear Time, Constant Space)

```
PROCEDURE RotateReverse(Vect, k)
```

```
n <- length(Vect)
```

```
k <- k mod n
```

```
Reverse(Vect, 0, n - 1)
```

```
Reverse(Vect, 0, k - 1)
```

```
Reverse(Vect, k, n - 1)
```

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

Complexity Analysis Questions:

Exercise 6: